

Algorithmes

Muhammad ibn Musa al-Khwarizmi (vers 780 – 850) était un mathématicien, astronome et géographe persan, dont les travaux ont profondément influencé le développement des mathématiques en Occident. Son ouvrage le plus célèbre, *Kitab al-Jabr wa-l-Muqabala*, introduit les méthodes systématiques pour résoudre des équations linéaires et quadratiques, donnant naissance au mot « algèbre » (al-jabr). Mais al-Khwarizmi est également célèbre pour ses méthodes de calcul : il décrivait des procédures étape par étape pour résoudre des problèmes arithmétiques et algébriques. Ces procédures sont aujourd'hui reconnues comme les premiers algorithmes, terme dérivé de la latinisation de son nom (Algoritmi).

Nicolas VACELET – Université Marie et Louis Pasteur

14 Octobre 2025

TD : Ordres de grandeur et complexité algorithmique

1. Ordres de grandeur

La notion de grandeur permet de comparer rapidement l'impact des différentes fonctions sur le temps de calcul ou la taille des données.

1. Comparer 10^{100} et 100^{10} . Lequel est le plus grand ?
2. Combien de chiffres comporte 2^n en écriture décimale ?
3. Estimer l'ordre de grandeur de $n!$ pour $n = 20$ et le comparer à 2^n et n^n .
4. Vérifier si n^5 est négligeable devant 2^n pour $n = 50$.

2. Comparaison de complexités

On s'intéresse à la hiérarchie des fonctions classiques.

1. Vérifier si $3^n = O(2^n)$ et justifier.
2. Vérifier si $\log 3^n = O(\log 2^n)$ et justifier.
3. Montrer que $n^2 = O(n^3)$ mais que $n^3 \neq O(n^2)$.
4. Comparer 2^n et $n!$ pour n grand.

3. Impact de la vitesse du processeur

Si votre programme peut résoudre un problème de taille maximale M en 1 minute, que se passe-t-il si le nouveau microprocesseur est 100 fois plus rapide ?

- Pour un algorithme linéaire $O(n)$
- Pour un algorithme quadratique $O(n^2)$
- Pour un algorithme de complexité $O(n^k)$
- Pour un algorithme exponentiel $O(2^n)$
- Question supplémentaire : que se passe-t-il pour un algorithme $O(n!)$?

4. Choix optimal entre algorithmes

Nous avons trois algorithmes avec complexités respectives $2n^2$, $10n$ et $40 \log_2 n$.

Discussion : Pour des petites valeurs de n , l'algorithme quadratique peut être acceptable, mais dès que n augmente, les algorithmes linéaire et logarithmique deviennent plus efficaces. Le choix dépend du contexte et de la taille des données.

1. Pour $n = 10$, quel est l'algorithme le plus rapide ?
2. Pour $n = 1000$, quel est l'algorithme le plus efficace ?
3. Quelle stratégie utiliseriez-vous pour choisir l'algorithme en fonction de la taille n ?

5. Variabilité de la complexité

Peut-on avoir un algorithme dont le pire cas est exponentiel, mais dont certaines entrées sont traitées rapidement ?

- Oui, certaines entrées peuvent être très simples, permettant une exécution en $O(n)$ pour ces cas.
- Non, si l'on exige $O(n)$ pour toutes les entrées, la complexité dans le pire cas ne peut plus être $O(2^n)$.
- Question supplémentaire : proposer un exemple concret d'algorithme avec un tel comportement.

6. Combinaison de sommes

Considérons la fonction suivante :

```
fonction f(n: entier): entier
début
  somme <- 0
  pour i variant de 0 à n
    pour j variant de 0 à (n-i)/2
      pour k variant de 0 à (n-i-2j)/5
        si (i+2j+5k = n) alors
          somme <- somme + 1
      retourner somme
fin
```

- Cette fonction compte le nombre de solutions entières (i, j, k) de l'équation $i + 2j + 5k = n$.
- Complexité : $O(n^3)$, car trois boucles imbriquées.
- Optimisation possible : inverser l'ordre des boucles et réduire la plus interne en utilisant une formule fermée $k = (n - i - 2j)/5$, ce qui réduit la complexité à $O(n^2)$.
- Exercice supplémentaire : modifier la fonction pour ne compter que les solutions avec $i \leq j \leq k$.